

Assessment Tool 1 — Internet Programming (CSA43)

CO-2: CSS Layout, Styling & JavaScript Interactivity — Full Solutions

Experiment 1: Responsive Layout Design

Scenario: A retail website is not responsive across mobile and tablet devices, causing misalignment of content and improper spacing.

(a) Analyze the possible reasons for lack of responsiveness

#	Root Cause	Effect
1	Missing <meta name="viewport"> tag	Mobile browsers render the page at desktop width (~980px) and scale it down
2	Fixed pixel widths (width: 960px) instead of fluid units (% , fr, vw)	Content overflows or leaves large gaps on smaller screens
3	No CSS media queries	Layout never adapts breakpoints for mobile/tablet/desktop
4	Table-based or float-based layouts	Floats break/collapse and require manual clearing, causing overlap
5	Fixed font sizes in px	Text becomes too large/small relative to the viewport
6	Images without max-width: 100%	Images overflow their containers on narrow screens
7	Fixed-height containers	Content is clipped when text wraps to more lines on small screens
8	box-sizing: content-box (default) with padding/border	Actual rendered width exceeds the declared width, causing overlap
9	No testing across real device breakpoints	Bugs are only caught after users report them

(b) Design a responsive webpage layout using CSS Grid

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Retail Store</title>
  <link rel="stylesheet" href="style.css" />
```

```

</head>
<body>
  <div class="page">
    <header class="site-header">
      <div class="logo">ShopEase</div>
      <nav class="site-nav">
        <a href="#">Home</a>
        <a href="#">Categories</a>
        <a href="#">Deals</a>
        <a href="#">Cart</a>
      </nav>
    </header>

    <main class="site-content">
      <aside class="sidebar">
        <h3>Filters</h3>
        <p>Price, Brand, Rating...</p>
      </aside>
      <section class="products">
        <article class="card">Product 1</article>
        <article class="card">Product 2</article>
        <article class="card">Product 3</article>
        <article class="card">Product 4</article>
      </section>
    </main>

    <footer class="site-footer">
      <p>&copy; 2026 ShopEase. All rights reserved.</p>
    </footer>
  </div>
</body>
</html>

```

style.css

```

* { box-sizing: border-box; margin: 0; padding: 0; }

.page {
  display: grid;
  grid-template-rows: auto 1fr auto;
  min-height: 100vh;
}

/* Header */
.site-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 1rem 1.5rem;
  background: #1f2937;
}

```

```

    color: #fff;
}
.site-nav { display: flex; gap: 1.2rem; }
.site-nav a { color: #fff; text-decoration: none; }

/* Main area: sidebar + product grid */
.site-content {
  display: grid;
  grid-template-columns: 220px 1fr;
  gap: 1rem;
  padding: 1rem;
}
.sidebar { background: #f3f4f6; padding: 1rem; border-radius: 8px; }

.products {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(180px, 1fr));
  gap: 1rem;
}
.card {
  background: #fff;
  border: 1px solid #e5e7eb;
  border-radius: 8px;
  padding: 1rem;
  text-align: center;
}

/* Footer */
.site-footer {
  text-align: center;
  padding: 1rem;
  background: #1f2937;
  color: #fff;
}

```

This uses **CSS Grid** for the page skeleton (header/content/footer rows) and an **auto-fill/minmax** product grid that reflows its column count automatically as the viewport shrinks — no JavaScript needed.

(c) Adapting the layout with media queries

```

/* Tablet: stack sidebar above products, keep nav horizontal */
@media (max-width: 900px) {
  .site-content {
    grid-template-columns: 1fr;
  }
  .sidebar { order: 2; }
  .products { order: 1; }
}

```

```

/* Mobile: stack header items, single-column product grid */
@media (max-width: 600px) {
  .site-header {
    flex-direction: column;
    gap: 0.6rem;
  }
  .site-nav { flex-wrap: wrap; justify-content: center; }
  .products {
    grid-template-columns: 1fr;
  }
}

```

- **Desktop (>900px):** two-column layout — sidebar (220px) + product grid.
- **Tablet (601px–900px):** the grid collapses to a single column; sidebar moves below the product list using order.
- **Mobile (≤600px):** header switches to a vertical stack, nav links wrap, and products display one per row for easy thumb scrolling.

Because the product grid already uses `repeat(auto-fill, minmax(180px, 1fr))`, it also reflows *within* each breakpoint without extra media queries — the explicit queries only handle structural changes (stacking sidebar/header).

(d) Spacing, alignment, and distribution techniques

- **box-sizing: border-box** applied globally so padding/border are included in declared widths — prevents the overlap described in the scenario.
- **Flexbox** (`display: flex; justify-content: space-between; align-items: center;`) keeps the logo and nav vertically centered and evenly distributed in the header.
- **CSS Grid gap** replaces manual margins between grid/flex children, giving consistent spacing without collapsing-margin bugs.
- **minmax() + auto-fill** in the product grid distributes leftover space evenly across cards instead of leaving a ragged last row.
- **Consistent spacing scale** (e.g., multiples of 0.5rem: 0.5rem, 1rem, 1.5rem...) keeps padding/margins visually rhythmic across sections.
- **text-align and order** used to reflow and re-align content (sidebar/footer) without changing the HTML source order, keeping markup accessible while presentation adapts.

(e) Evaluation and further improvements

Effectiveness: The Grid + Flexbox + media-query approach removes the fixed-width/overlap issues identified in (a): the layout now scales fluidly, the viewport meta tag ensures mobile browsers render at true device width, and `box-sizing: border-box` eliminates box-model overflow.

Suggested improvements: 1. Replace fixed breakpoints with **fluid typography** using `clamp(1rem, 2vw + 0.5rem, 1.5rem)` so text scales continuously rather than jumping at

breakpoints. 2. Adopt **CSS Container Queries** (@container) for the product card component so cards adapt to the width of their *container* (sidebar vs. full-width) rather than only the viewport. 3. Add a **hamburger menu** for the nav below 480px instead of wrapping links. 4. Test with **Chrome DevTools device toolbar**, **BrowserStack**, and Lighthouse's mobile-friendliness audit. 5. Use **srcset/sizes** on product images to serve appropriately sized images per device, improving both layout stability and performance.

Experiment 2: Form Validation using JavaScript

Scenario: A registration form accepts invalid email and phone number inputs, leading to incorrect data submission.

(a) Analyze the issues in the existing validation mechanism

- Fields likely use generic `type="text"`, so the browser applies **no built-in format checking**.
- No **client-side JavaScript validation** exists, so malformed data (e.g., `abc@`, `12ab34`) reaches the server unfiltered.
- No **real-time feedback** — users only discover errors after full submission (or not at all if the server doesn't validate either), hurting UX.
- Lack of **regular-expression pattern matching** for structured fields like email/phone means format errors (missing @, wrong digit count, invalid country code) are not caught.
- No distinction between **required vs. optional** fields, and no `maxlength/inputmode` hints for mobile keyboards.
- Relying only on client-side checks (if any) is also a **security gap** — validation must be duplicated server-side, since client JS can be bypassed.

(b) Design input fields with appropriate HTML attributes

```
<form id="registerForm" novalidate>
  <div class="field">
    <label for="email">Email Address</label>
    <input
      type="email"
      id="email"
      name="email"
      placeholder="you@example.com"
      required
      autocomplete="email"
      maxlength="100"
    />
    <span class="error" id="emailError"></span>
  </div>

  <div class="field">
```

```

<label for="phone">Phone Number</label>
<input
  type="tel"
  id="phone"
  name="phone"
  placeholder="9876543210"
  required
  pattern="[0-9]{10}"
  inputmode="numeric"
  autocomplete="tel"
  maxlength="10"
/>
<span class="error" id="phoneError"></span>
</div>

<button type="submit">Register</button>
</form>

```

type="email" and type="tel" trigger the correct mobile keyboard; required, pattern, and maxlength give a first layer of native HTML5 validation, which is then reinforced by JavaScript for more precise, custom error messages.

(c) JavaScript validation using Regular Expressions

```

const emailRegex = /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/;
const phoneRegex = /^[6-9]\d{9}$/; // 10-digit, starts 6-9 (India-style mobile numbers)

```

```

function validateEmail(value) {
  return emailRegex.test(value.trim());
}

```

```

function validatePhone(value) {
  return phoneRegex.test(value.trim());
}

```

- The **email regex** enforces: one or more allowed local-part characters, an @, a domain with at least one dot, and a 2+ letter TLD.
- The **phone regex** enforces exactly 10 digits, with the Indian mobile convention that the first digit is 6–9 (adjust the pattern for other locales/country codes as needed).

(d) Real-time error message mechanism

```

const form = document.getElementById('registerForm');
const emailInput = document.getElementById('email');
const phoneInput = document.getElementById('phone');
const emailError = document.getElementById('emailError');
const phoneError = document.getElementById('phoneError');

function showError(inputEl, errorEl, message) {
  inputEl.classList.add('invalid');
}

```

```

    errorEl.textContent = message;
}

function clearError(inputEl, errorEl) {
    inputEl.classList.remove('invalid');
    errorEl.textContent = '';
}

emailInput.addEventListener('input', () => {
    if (emailInput.value === '' || validateEmail(emailInput.value)) {
        clearError(emailInput, emailError);
    } else {
        showError(emailInput, emailError, 'Enter a valid email address (e.g. name@domain.com)');
    }
});

phoneInput.addEventListener('input', () => {
    if (phoneInput.value === '' || validatePhone(phoneInput.value)) {
        clearError(phoneInput, phoneError);
    } else {
        showError(phoneInput, phoneError, 'Enter a valid 10-digit mobile number');
    }
});

form.addEventListener('submit', (e) => {
    e.preventDefault();
    let valid = true;

    if (!validateEmail(emailInput.value)) {
        showError(emailInput, emailError, 'Enter a valid email address');
        valid = false;
    }
    if (!validatePhone(phoneInput.value)) {
        showError(phoneInput, phoneError, 'Enter a valid 10-digit mobile number');
        valid = false;
    }

    if (valid) {
        alert('Registration successful!');
        form.reset();
    }
});

.invalid { border: 1px solid #dc2626; background: #fef2f2; }
.error { color: #dc2626; font-size: 0.85rem; display: block; min-height: 1.1em; }

```

The input event listener fires on every keystroke, giving **immediate, field-level feedback** as the user types, while the submit handler performs a final full-form check and blocks submission (`preventDefault`) if any field is still invalid.

(e) Evaluation and enhancements

Effectiveness: Combining HTML5 attributes with JS regex validation catches malformed emails/phone numbers before they leave the browser, and real-time feedback (via the input event) prevents users from reaching the submit button with unnoticed errors.

Suggested enhancements: 1. **Never trust the client alone** — replicate the same email/phone checks on the server, since JavaScript can be disabled or bypassed. 2. **Debounce** the input handler (e.g., 300ms) to avoid showing an error message while the user is still mid-typing. 3. Add **ARIA attributes** (`aria-invalid`, `aria-describedby`) so screen readers announce errors, improving accessibility. 4. Support **international phone formats** using a library like `libphonenumber-js` instead of a single fixed-length regex. 5. Add a **confirm-email** field or an OTP/verification-link step to reduce typos that pass regex but are still wrong addresses. 6. Rate-limit/CAPTCHA the submit endpoint server-side to prevent bot registrations, since regex validation alone doesn't stop automated abuse.

Experiment 3: CSS Layout Debugging

Scenario: A webpage layout overlaps when the browser window is resized, affecting usability.

(a) Analyze the CSS properties responsible for overlapping

Property / Pattern	How it causes overlap on resize
<code>position: absolute</code> without a properly positioned ancestor	Element is positioned relative to the viewport/body instead of its intended container, so it drifts over other content as the window resizes
Fixed width/height in px	Doesn't shrink with the viewport, so siblings that <i>do</i> shrink end up overlapping the fixed element
Floated elements without clear	Following content flows up beside/under the float instead of below it once wrapping changes
<code>box-sizing: content-box</code> (default) + padding/border	Rendered width = declared width + padding + border, so elements are wider than expected and collide with neighbors
Negative margin used for "quick" alignment fixes	Pulls elements over their neighbors; breaks at different widths
Missing flex-wrap on a flex container	Items are forced to shrink or overflow instead of wrapping to a new line
z-index stacking without	Elements pile on the same stacking layer unpredictably

Property / Pattern	How it causes overlap on resize
position context	as layout shifts

(b) Role of the CSS box model in the layout

The box model determines how much space each element actually occupies: **content** → **padding** → **border** → **margin**, from innermost to outermost.

- **content-box (default):** width/height apply only to the content area; padding and border are added *on top*, so the true rendered size is larger than the declared width. This is a common cause of unexpected overlap when several elements are placed side-by-side with declared widths that “should” sum to 100%.
- **border-box:** width/height include padding and border, so an element with width: 25% and padding: 1rem still occupies exactly 25% — this is why box-sizing: border-box is the standard reset for predictable layouts.
- **margin:** creates space *between* boxes; adjacent vertical margins can collapse, which can cause spacing to look inconsistent if not accounted for.
- Understanding this model is essential before debugging overlap, because “overlap” is very often just the box model making an element wider/taller than the developer assumed.

(c) Debugging using positioning techniques

Before (buggy) — float + fixed width + content-box:

```
.sidebar { float: left; width: 250px; padding: 20px; } /* content-box → true width 290px */
.main    { float: left; width: 750px; padding: 20px; } /* content-box → true width 790px */
/* 290px + 790px = 1080px, overflows a 1000px container → overlap/wrap */
```

Debug steps: 1. Open DevTools → inspect each element’s computed box model to see the *true* rendered width (content + padding + border). 2. Confirm box-sizing on the offending elements — likely still content-box. 3. Check whether floats are cleared after the row; missing clearance is a common companion bug. 4. Look for any position: absolute/relative mismatches by inspecting the nearest positioned ancestor.

After (fixed) — Flexbox with border-box:

```
* { box-sizing: border-box; }

.layout {
  display: flex;
  flex-wrap: wrap; /* allows items to drop to a new row instead of overlapping */
  gap: 20px;
}
.sidebar { flex: 0 0 250px; padding: 20px; }
```

```
.main { flex: 1 1 600px; padding: 20px; min-width: 0; } /* min-width:0 prevents flex overflow */
```

Switching from **float** to **Flexbox**, adding `flex-wrap: wrap`, and applying `box-sizing: border-box` resolves the overlap: the box model is now predictable, and the layout gracefully wraps instead of forcing elements into the same space when the window shrinks.

(d) Modified CSS — final corrected layout

```
* { box-sizing: border-box; margin: 0; padding: 0; }

.layout {
  display: flex;
  flex-wrap: wrap;
  gap: 1.25rem;
  padding: 1.25rem;
}

.sidebar {
  flex: 1 1 250px;
  background: #f3f4f6;
  padding: 1rem;
  border-radius: 8px;
}

.main {
  flex: 3 1 600px;
  min-width: 0;
  background: #fff;
  padding: 1rem;
  border-radius: 8px;
}

/* Alternative equally valid fix using Grid */
.layout--grid {
  display: grid;
  grid-template-columns: minmax(200px, 1fr) 3fr;
  gap: 1.25rem;
}

@media (max-width: 700px) {
  .layout--grid { grid-template-columns: 1fr; }
}
```

Both the Flexbox (`flex-wrap` + `flex-basis`) and Grid (`minmax()` + a responsive `grid-template-columns`) versions eliminate the overlap by letting the browser calculate space distribution instead of relying on hard-coded pixel widths.

(e) Evaluation and best practices

Effectiveness: The corrected layout no longer overlaps at any width — items either shrink proportionally (flex/fr units) or wrap to a new row (flex-wrap: wrap), and box-sizing: border-box ensures declared widths match rendered widths.

Best practices to maintain consistency: 1. Set box-sizing: border-box globally as a reset — the single highest-impact fix for box-model overlap bugs. 2. Prefer **Flexbox/Grid** over floats and manual position: absolute for structural layout; reserve absolute positioning for genuinely overlapping UI (tooltips, badges, modals). 3. Use **relative units** (% , fr, minmax(), clamp()) instead of fixed px widths for anything that must adapt. 4. Always pair float (if still used for legacy reasons) with a proper **clearfix**. 5. Add flex-wrap: wrap (Flexbox) or a responsive grid-template-columns (Grid) so content reflows instead of overlapping at narrow widths. 6. Regularly test resizing behavior with **DevTools' responsive mode** across common breakpoints (320px, 768px, 1024px, 1440px) as part of the development workflow, not just at the end.

Code of Conduct

I **MANDLA VISWATEJA** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____